IR Systém

Automatická indexace a vyhledávání dokumentů

Semestrální práce — KIV/IR

Západočeská univerzita v Plzni

Jazyk implementace:	Python 3.10
Datová sada:	Wowhead News Articles (711 dok.) Evaluační kolekce (609 dok.)
Modely vyhledávání:	TF-IDF, Boolean, LSA, SentenceTransformers

11. května 2026

Obsah

1 Úvod **3**

2 Architektura systému **3**

3 Předzpracování textu **3**
3.1 Tokenizace . . . 3
3.2 Normalizace . . . 4
3.3 Filtrace a morfologické zpracování . . . 4

4 Invertovaný index **4**
4.1 TF-IDF index . . . 4
4.2 Boolean index . . . 5

5 Vyhledávání **5**
5.1 Vektorový model (TF-IDF) . . . 5
5.2 Booleovský model — vlastní parser dotazů . . . 5
5.2.1 Fáze 1: Tokenizace dotazu . . . 5
5.2.2 Fáze 2: Převod na RPN (Shunting-Yard algoritmus) . . . 6
5.2.3 Fáze 3: Stavba AST a vyhodnocení . . . 6
5.3 Sémantický model — LSA . . . 6
5.4 SentenceTransformers (BERT) . . . 7

6 Grafické rozhraní (GUI) **7**
6.1 Záložka Indexace . . . 7
6.2 Záložka Vyhledávání . . . 7
6.3 Záložka Evaluace . . . 8

7 Evaluace **8**
7.1 Použité metriky . . . 8
7.2 Podmínky evaluace . . . 8
7.3 Výsledky . . . 9
7.4 Rychlost . . . 9

8 Nadstandardní funkčnosti **9**
8.1 Vlastní implementace parsování dotazů bez externí knihovny . . . 9
8.2 Více scoring modelů . . . 10
8.3 Implementace sémantických modelů (LSA a Transformers) . . . 10
8.4 Pozdější doindexování dat . . . 10
8.5 Grafické rozhraní (GUI) . . . 10
8.6 Zvýraznění hledaného textu v náhledu výsledků . . . 11
8.7 Vylepšení vyhledávání — optimalizace boolean indexu . . . 11
8.8 Ošetření HTML tagů . . . 11
8.9 Další předzpracování a normalizace . . . 11

9 Závěr **12**

A Struktura souborů **12**

B Příklady dotazů **12**

1 Úvod

Cílem semestrální práce je implementace vlastního systému automatické indexace a vyhledávání dokumentů. Systém podporuje dva primární datové zdroje: kolekci článků ze serveru Wowhead.com získaných vlastním crawlerem a standardní evaluační kolekci.

Klíčové vlastnosti systému:

- Tři modely vyhledávání: vektorový (TF-IDF), booleovský a sémantický (LSA)
- Vlastní parser booleovských dotazů bez použití externích knihoven
- Grafické rozhraní (GUI) s podporou doindexování a zvýrazňování textu
- Evaluace pomocí metriky MAP

2 Architektura systému

Systém je rozdělen do pěti modulů:

Modul	Soubor	Účel
Předzpracování	`preprocessing.py`	Tokenizace, normalizace, stemming/lemmatizace
TF-IDF index	`tfidf_search.py`	Invertovaný index, TF-IDF váhy, cosine similarity
Boolean index	`boolean_search.py`	Parser dotazů, AST, množinové operace
Evaluace / LSA	`eval_ir.py`	LSA (SVD), SentenceTransformers, MAP metrika
GUI	`gui.py`	Grafické rozhraní (tkinter)

Data z obou kolekcí jsou nezávisle načtena, tokenizována a zaindexována do oddělených pojmenovaných indexů. Každý index obsahuje souběžně TF-IDF index, booleovský index a (lazily budovaný) LSA index.

3 Předzpracování textu

3.1 Tokenizace

Tokenizace je implementována pomocí regulárního výrazu, který správně zpracovává URL adresy, složená slova s pomlčkou a čísla:

```
tokens = re.findall(
    r'(?:https?://\S+)|(?:\w+(?:[-]\w+)*)|(?:\d+)',
    text.lower()
)
```

Listing 1: Regex tokenizace v preprocessing.py

3.2 Normalizace

Systém provádí následující normalizační kroky **před** tokenizací:

- **Odstranění HTML tagů** — regulárním výrazem `<[^>]+>`
- **Normalizace dat** — datum ve formátu `2024/03/15` je převedeno na slovní formu `march fifteenth two thousand and twenty-four` pomocí knihovny `num2words`
- **Čísla na slova** — `42` → `forty-two`
- **Odstranění diakritiky** — pomocí knihovny `unidecode`

3.3 Filtrace a morfologické zpracování

Po tokenizaci jsou aplikovány tyto kroky:

1. Odstranění stopwords (NLTK anglický seznam)
2. Filtrace tokenů kratších než 2 znaky
3. Volitelně: **stemming** (Porter Stemmer) nebo **lemmatizace** (WordNet Lemmatizer)

Stemming a lemmatizace jsou vzájemně exkluzivní a konfigurovatelné přes GUI. Pro evaluaci byl použit WordNet lemmatizer, který dosahuje lepších výsledků.

4 Invertovaný index

4.1 TF-IDF index

TF-IDF index (`TfidfIndex`) je implementován jako in-memory invertovaný index s následující strukturou:

$$\text{postings}[\text{term}] = \{\text{doc_index} \mapsto \text{tfidf_weight}\}$$

Výpočet vah:

$$\text{TF}(t, d) = 1 + \log_{10}(\text{count}(t, d))$$

$$\text{IDF}(t) = \log_{10}\left(\frac{N}{\text{DF}(t)}\right)$$

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Podobnost dotazu a dokumentu je měřena **cosine similarity** na L2-normalizovaných vektorech:

$$\text{sim}(q, d) = \frac{\vec{q} \cdot \vec{d}}{\|\vec{q}\| \cdot \|\vec{d}\|}$$

Optimalizace vyhledávání: Systém nevyhodnocuje všechny dokumenty, ale pomocí invertovaného indexu nejprve identifikuje kandidátní dokumenty (union postings listů všech termů dotazu) a ohodnotí pouze je.

4.2 Boolean index

Booleovský index (`BooleanIndex`) ukládá postings jako množiny (frozenset) doc_id:

$$\text{postings}[\text{term}] = \text{frozenset}(\{d \mid t \in d\})$$

Implementované optimalizace:

- `A AND NOT B` → množinový rozdíl $A \setminus B$ (bez materializace $\neg B$)
- Short-circuit AND: prázdný levý operand ⇒ okamžitě vrátí $\emptyset$
- Short-circuit OR: levý operand $= \mathcal{U}$ ⇒ okamžitě vrátí $\mathcal{U}$
- Reordering AND: vždy se iteruje přes menší množinu

5 Vyhledávání

5.1 Vektorový model (TF-IDF)

Dotaz je předzpracován stejným způsobem jako dokumenty, převeden na TF-IDF vektor a porovnán s dokumenty pomocí cosine similarity. Výsledky jsou seřazeny sestupně dle skóre.

5.2 Booleovský model — vlastní parser dotazů

Toto je klíčová nadstandardní funkčnost. Parser booleovských dotazů je implementován **zcela bez externích knihoven** ve třech fázích:

5.2.1 Fáze 1: Tokenizace dotazu

Regulární výraz rozpozná operátory (AND, OR, NOT), závorky a termíny:

```
pattern = re.compile(
    r"\(|\)|\bAND\b|\bOR\b|\bNOT\b|[A-Za-z0-9_]+(?:-[A-Za-z0-9_
        ]+)*"
)
```

Listing 2: Tokenizace boolean dotazu

5.2.2 Fáze 2: Převod na RPN (Shunting-Yard algoritmus)

Infixový výraz je převeden na reverzní polskou notaci (RPN) pomocí **Shunting-Yard algoritmu** (E. W. Dijkstra). Priorita operátorů:

Operátor	Priorita	Asociativita
NOT	3	pravá
AND	2	levá
OR	1	levá

5.2.3 Fáze 3: Stavba AST a vyhodnocení

Z RPN je rekurzivně postaven **abstraktní syntaktický strom** (AST), jehož uzly jsou datové třídy s atributy `op` (TERM / AND / OR / NOT), `left` a `right`. Strom je vyhodnocen rekurzivní funkcí `_eval_node`.

```
if node.op == "AND":
    # Optimalizace: A AND (NOT B) -> A - B
    if node.right.op == "NOT":
        return self._eval_node(node.left) - \
               self._eval_node(node.right.left)
    left = self._eval_node(node.left)
    if not left:            # short-circuit
        return set()
    right = self._eval_node(node.right)
    if len(left) > len(right):
        left, right = right, left   # reorder
    return left & right
```

Listing 3: Příklad: vyhodnocení AND uzlu s optimalizacemi

Před odesláním dotazu do indexu jsou termy dotazu předzpracovány stejnou metodou jako dokumenty (lowercase, stemming/lemmatizace), aby bylo zajištěno správné párování.

5.3 Sémantický model — LSA

Latentní sémantická analýza (LSA / LSI) je implementována pomocí oříznutého SVD rozkladu TF-IDF matice:

$$A_{m\times n} \approx U_{m\times k} \cdot \Sigma_{k\times k} \cdot V^T_{k\times n}$$

kde m je velikost slovníku, n počet dokumentů a $k = 200$ je počet latentních dimenzí. Dotaz je promítnut do latentního prostoru:

$$\vec{q}_{\text{lat}} = \frac{\vec{q} \cdot U}{\Sigma}$$

a porovnán s dokumenty pomocí cosine similarity v latentním prostoru. SVD je počítán pomocí `scipy.sparse.linalg.svds` (Lanczosova metoda pro řídké matice).

LSA index je budován **lazily** při prvním dotazu a následně cachován v paměti. Tím se zamezí zbytečnému výpočtu SVD pro uživatele, kteří LSA model nevyužívají.

5.4 SentenceTransformers (BERT)

Pro evaluaci je dostupný i model `all-MiniLM-L6-v2` (SentenceTransformers), který kóduje dokumenty a dotazy do hustých vektorů pomocí předtrénovaného transformerového modelu. Podobnost je měřena cosine similarity v embedding prostoru.

6 Grafické rozhraní (GUI)

GUI je implementováno pomocí knihovny `tkinter` (ttk) bez dalších závislostí. Okno je rozděleno do tří záložek.

6.1 Záložka Indexace

Umožňuje:

- Výběr datového zdroje: Wowhead articles, evaluační data nebo vlastní soubor
- Konfiguraci předzpracování (lemmatizace / stemming / diakritika)
- Pojmenování indexu — lze mít libovolný počet indexů najednou
- Zobrazení statistik indexu (počet dokumentů, velikost slovníku, čas indexace)

Doindexování (nadstandardní funkčnost): Sekce „Doindexovat“ umožňuje přidat nová data do existujícího indexu bez nutnosti opětovného načítání původních dat. Duplikáty (stejné doc_id) jsou automaticky přeskočeny. Boolean index je aktualizován přírůstkově (`add_documents`), TF-IDF index je přestaven z kombinovaných tokenů (původní + nové). Cachovaný LSA index je invalidován.

6.2 Záložka Vyhledávání

Uživatel zadá dotaz a zvolí:

- **Index** — který pojmenovaný index použít
- **Model** — TF-IDF (vektorový) / Boolean (AND/OR/NOT) / LSA (sémantické)
- **Top-K** — počet výsledků (výchozí: 10)

Výsledky jsou zobrazeny v tabulce s pořadím, skóre, doc_id a názvem. Status bar zobrazuje celkový počet nalezených dokumentů, čas vyhledávání a použitý model.

Zvýraznění textu (nadstandardní funkčnost): Po kliknutí na výsledek se zobrazí náhled dokumentu. Slova z dotazu jsou automaticky zvýrazněna žlutě v celém textu. Pro booleovský model jsou zvýrazněny termíny dotazu (operátory AND/OR/NOT jsou ignorovány). Zvýraznění je case-insensitive.

6.3 Záložka Evaluace

Umožňuje spuštění evaluace na libovolném indexu s libovolným souborem dotazů. Výsledky (MAP@10, MAP@20, MAP@50) jsou zobrazeny v logu. Výsledky lze exportovat ve TREC formátu (`qid iter docno rank sim run_id`) pro spuštění externího evaluačního nástroje.

7 Evaluace

7.1 Použité metriky

Average Precision (AP) pro jeden dotaz:

$$\mathrm{AP} = \frac{1}{|\mathrm{Rel}|} \sum_{k=1}^{n} P(k) \cdot \mathrm{rel}(k)$$

kde $P(k)$ je precision na pozici k a $\mathrm{rel}(k) \in \{0, 1\}$ indikuje relevanci dokumentu na pozici k.

Mean Average Precision (MAP) je průměr AP přes všechny dotazy:

$$\mathrm{MAP} = \frac{1}{|Q|} \sum_{q \in Q} \mathrm{AP}(q)$$

MAP@K zohledňuje pouze prvních K výsledků.

7.2 Podmínky evaluace

- **Kolekce:** 609 evaluačních dokumentů (`data/documents.json`)
- **Dotazy:** 100 dotazů s relevančními posudky (`data/full_text_queries.json`)
- **Předzpracování:** WordNet lemmatizace + odstranění diakritiky + stopwords

7.3 Výsledky

Výsledky evaluace

Model	MAP@10	MAP@20	MAP@50
TF-IDF (vektorový)	**0.5198**	—	—
LSA (sémantický, $k = 200$)	**0.3895**	—	—

TF-IDF dosahuje skóre **0.5198**, což je více než 2,5× nad požadovanou hodnotou 0.20. LSA s 200 latentními dimenzemi dosahuje **0.3895**, téměř 2× nad cílem.

TF-IDF překonává LSA na této kolekci z očekávaného důvodu: evaluační kolekce má 609 dokumentů, kde přesná shoda termů (TF-IDF) funguje lépe než latentní reprezentace (LSA), která svůj potenciál plně projeví na rozsáhlejších kolekcích s bohatší synonymií.

7.4 Rychlost

Operace	Wowhead (711 dok.)	Eval (609 dok.)
Indexace (TF-IDF + Boolean)	∼3–5 s	∼2–3 s
Vyhledávání TF-IDF (1 dotaz)	<10 ms	<5 ms
Vyhledávání Boolean (1 dotaz)	<5 ms	<3 ms
LSA build (SVD)	∼10 s	∼10 s
Vyhledávání LSA (po LSA build)	<5 ms	<5 ms

8 Nadstandardní funkčnosti

Tato sekce popisuje implementované nadstandardní funkčnosti seřazené dle kategorií ze zadání.

8.1 Vlastní implementace parsování dotazů bez externí knihovny

Celý parser booleovských dotazů (viz sekci 5.2) je implementován **zcela bez externích knihoven** pro parsování (bez `pyparsing`, `ply`, `lark` apod.). Implementace zahrnuje čtyři komponenty vlastního návrhu:

- **Lexer** — regulární výraz rozpoznávající operátory, závorky a termíny
- **Shunting-Yard algoritmus** — převod infixového výrazu na RPN (E. W. Dijkstra)
- **Stavba AST** — rekurzivní budování abstraktního syntaktického stromu ze zásobníku
- **Vyhodnocení AST** — rekurzivní interpreter s množinovými optimalizacemi

8.2 Více scoring modelů

Systém implementuje **čtyři** modely vyhledávání, přičemž všechny jsou dostupné jak ve vyhledávacím rozhraní GUI, tak v evaluačním modulu:

Model	Typ	Search tab	Eval tab
TF-IDF + cosine similarity	Vektorový	✓	✓
Boolean (AND/OR/NOT)	Exaktní	✓	—
LSA ($k = 200$, SVD)	Sémantický	✓	✓
SentenceTransformers (BERT)	Neuronový	✓	✓

8.3 Implementace sémantických modelů (LSA a Transformers)

LSA (Latentní sémantická analýza): Vlastní implementace v `eval_ir.py` využívá řídkou TF-IDF matici a oříznutý SVD rozklad (`scipy.sparse.linalg.svds`, Lanczosova metoda) s $k = 200$ latentními dimenzemi. Dotaz je promítnut do latentního prostoru a porovnán s dokumenty pomocí cosine similarity.

SentenceTransformers (BERT): Model `all-MiniLM-L6-v2` kóduje dokumenty i dotazy do hustých vektorů (384 dimenzí) pomocí předtrénovaného transformeru. Podobnost je měřena jako cosine similarity v embedding prostoru.

Oba modely jsou budovány **lazily** při prvním dotazu a výsledné reprezentace (SVD matice / embeddingy) jsou **cachovány v paměti** — každý další dotaz je okamžitý.

8.4 Pozdější doindexování dat

GUI umožňuje přidat nová data do existujícího indexu bez nutnosti opětovného načítání dat z disku. Uživatel vybere cílový index a soubor s novými dokumenty. Systém automaticky:

- Přeskočí duplicitní dokumenty (detekce podle doc_id)
- Přírůstkově aktualizuje Boolean index metodou `add_documents()`
- Přestaví TF-IDF index z paměti (spojení původních + nových tokenů) bez čtení z disku
- Invaliduje cachovaný LSA a SentenceTransformers index

8.5 Grafické rozhraní (GUI)

GUI implementováno v `tkinter/ttk` bez dalších závislostí, rozdělené do tří záložek:

- **Indexace** — výběr datového zdroje, konfigurace předzpracování, statistiky, doindexování
- **Vyhledávání** — výběr indexu a modelu, zadání dotazu, tabulka výsledků, náhled dokumentu

- **Evaluace** — výpočet MAP@10/20/50, export výsledků ve TREC formátu

Lze mít libovolný počet pojmenovaných indexů najednou a přepínat mezi nimi i mezi modely za běhu.

8.6 Zvýraznění hledaného textu v náhledu výsledků

Po kliknutí na výsledek v tabulce se zobrazí náhled dokumentu, kde jsou slova z dotazu automaticky zvýrazněna žlutým pozadím. Implementováno pomocí systému tagů widgetu `ScrolledText`. Zvýraznění je:

- **Case-insensitive** — „Dragon“ i „dragon“ se označí stejně
- **Model-aware** — pro Boolean dotaz jsou zvýrazněny pouze termíny (AND/OR/-NOT/závorky jsou ignorovány)
- Funguje pro všechny čtyři modely vyhledávání

8.7 Vylepšení vyhledávání — optimalizace boolean indexu

Boolean index obsahuje čtyři implementované optimalizace výkonu:

1. `A AND NOT B` $\rightarrow A \setminus B$ — set difference místo materializace doplňku $\neg B$
2. **Short-circuit AND** — prázdná levá množina $\Rightarrow$ okamžitě vrátí $\emptyset$ bez vyhodnocení pravé strany
3. **Short-circuit OR** — levá množina $= \mathcal{U} \Rightarrow$ okamžitě vrátí $\mathcal{U}$
4. **Reordering AND** — vždy se iteruje přes menší z operandů

TF-IDF vyhledávání je optimalizováno pomocí invertovaného indexu: hodnotí se pouze kandidátní dokumenty (union postings listů termů dotazu), nikoli celá kolekce.

8.8 Ošetření HTML tagů

Preprocessing automaticky odstraňuje HTML tagy před tokenizací pomocí regulárního výrazu:

```
text = re.sub(r'<[^>]+>', '', text)
```

To zajišťuje čistou indexaci dokumentů, které mohou obsahovat HTML markup (např. obsah stažený crawlerem).

8.9 Další předzpracování a normalizace

Nad rámec standardní tokenizace a stopwords jsou implementovány:

- **Normalizace dat** — datum `2024/03/15` $\rightarrow$ `march fifteenth two thousand and twenty-four` (knihovna `num2words`, regex detekce formátu)

- **Čísla na slova** — `42` → `forty-two`
- **Odstranění diakritiky** — `unidecode` (konfigurovatelné)
- **Zpracování složených slov s pomlčkou** — `world-first` zůstane jako jeden token
- **Zpracování URL** — URL adresy jsou zachovány jako celek, ne rozsekány na části

9 Závěr

Implementovaný IR systém splňuje veškerou minimální nutnou funkčnost zadání a přináší devět nadstandardních funkčností: vlastní parser booleovských dotazů (shunting-yard + AST bez externích knihoven), čtyři scoring modely (TF-IDF, Boolean, LSA, SentenceTransformers/BERT), doindexování, zvýraznění textu, vylepšení vyhledávání (optimalizace boolean indexu a kandidátní filtrování TF-IDF), ošetření HTML tagů, rozšířenou normalizaci (data, čísla, diakritika), zobrazení více polí dokumentu a GUI s tkinter.

Systém dosahuje **MAP skóre 0.52** na evaluační kolekci.

A Struktura souborů

Soubor	Popis
`gui.py`	Grafické rozhraní (tkinter)
`preprocessing.py`	Tokenizace, normalizace, stemming/lemmatizace
`tfidf_search.py`	TF-IDF index, cosine similarity
`boolean_search.py`	Boolean index, shunting-yard parser, AST
`eval_ir.py`	LSA, SentenceTransformers, MAP metrika
`crawler.py`	Selenium crawler pro Wowhead.com
`requirements.txt`	Závislosti projektu
`README.md`	Návod ke spuštění
`wowhead_articles.jsonl`	Wowhead data (711 článků)
`data/documents.json`	Evaluační dokumenty (609)
`data/full_text_queries.json`	Evaluační dotazy (100)
`data/gold_relevancies.txt`	Zlatý standard relevancí

B Příklady dotazů

TF-IDF a LSA

```
patch content
raid boss mechanics
player housing update
world of warcraft shadowlands
```

Boolean

```
warcraft AND shadowlands
(dragon OR raid) AND NOT pvp
NOT arena AND battleground OR guild
patch AND (warrior OR paladin) AND NOT nerf
```